

PyTorch CXL Dataset/DataLoader

驾驭数百TB DDR内存的分布式数据加载架构

扩展 PyTorch API · 显式利用 CXL 语义 · 数百 TB 内存池化

CXL Memory Pool

PyTorch DataLoader

MegaScale

FFCV

Hundreds TB

核心论点

通过扩展 PyTorch Dataset/DataLoader API，构建 CXL 内存池感知的分布式数据加载系统——不是透明方案，而是显式利用 CXL 语义（字节级寻址、NUMA 感知、分层放置），实现数百 TB 内存的全局数据池化、智能预取与负载均衡。

CXL 内存池

字节级寻址
NUMA 感知
分层放置

分布式数据加载

拓扑感知采样
分层全局 Shuffle
三级预取

数百 TB 驾驭

全局内存池
元数据分层
动态迁移

目录

01

背景与问题定义

大规模训练的内存墙

02

相关工作调研

MegaScale-Data / FFCV /
DeepSpeed

03

核心挑战

四大技术挑战

04

系统架构设计

整体架构与三层 API

05

关键模块设计

Pool Allocator / Dataset / Sampler

06

数百TB驾驭方案

全局内存池 / 分层放置

07

性能分析与预期收益

理论分析与加速比

01 背景与问题定义

大规模训练的内存墙

GPT-4 级别训练: ~13T tokens → ~50TB 原始文本 → ~100TB Tokenized → ~200TB 中间结果

LLaMA-405B 级别训练: ~20T tokens → ~100TB 原始文本 → ~300TB Tokenized → ~500TB 中间结果

单节点 DRAM: 0.5-2TB | 单节点 GPU 显存: 80-800GB | 数据/计算比: 100:1 ~ 1000:1

传统方案局限

PyTorch DataLoader 假设数据在本地磁盘

内存映射受限于本地 NUMA 节点

DALI 仅限 GPU 解码, 不管理全局内存池

CXL 机遇

将内存层次从「节点内」扩展到「机架级」

数百 TB 内存以字节寻址方式暴露给所有节点

CXL: 300ns 延迟, 50GB/s 带宽

02 相关工作调研

MegaScale-Data (2504.09844): 多源 LFM 训练数据加载

创新：解耦预处理 + 集中式数据平面 + 多级自动分区

性能：端到端训练吞吐提升 4.5x | CPU 内存占用降低 13.5x

启示：解耦预处理 → CXL 池中不同数据源可放置在不同 tier

FFCV (2306.12517): 消除数据瓶颈

创新：高效文件格式 + 缓存策略 + 异步传输 + JIT 编译

性能：ResNet-50 ImageNet 训练到 75% 仅 20 分钟

启示：缓存策略可扩展为三层：DRAM → CXL Pool → NVMe

DeepSpeed Data Efficiency (2212.03597): 高效数据采样与路由

创新：Curriculum Learning + Random Layerwise Token Dropping

性能：GPT-3 1.3B 预训练 12.5x 成本降低，保持 95% 模型质量

启示：Curriculum Learning 可结合 CXL 内存层次

03 核心挑战

挑战1 · 数据局部性 vs 全局随机访问

LLM 预训练需要全局 shuffle，但 CXL 有延迟不对称

- 分层 shuffle: 节点内随机，节点间顺序
- 预取缓冲 + 数据放置调整

挑战2 · 数百 TB 内存的元数据管理

100TB 数据 = 10^{13} tokens，元数据 80MB 可放入 DRAM

- 但变长序列 1 亿条 $\times$ 1KB = 100GB 元数据需分层
- Bloom filter 加速负查找

挑战3 · 跨节点的数据一致性

多节点同时读取同一 CXL 区域

- CXL 2.0 无硬件 cache coherence
- 只读数据共享，读写数据 ownership-based coherence

挑战4 · 异构硬件拓扑

不同节点 CXL 拓扑不同（直连 vs Switch vs RDMA）

- 拓扑感知的数据分区
- 根据节点能力动态调整数据量

04 系统架构设计

用户代码层

```
from cxl_dataloader import CXLDataset, CXLDataLoader
dataset = CXLDataset('s3://data/', pool='cxl_pool_A')
loader = CXLDataLoader(dataset, batch_size=32, pin_to_cxl=True)
```

CXL Dataset/DataLoader API 层

CXLDataset: `__getitem__` → CXL.mem 读取
CXLSampler: DistributedCXLSampler (拓扑感知)
CXLDataLoader: `pin_to_cxl` + `prefetch_to_gpu`

运行时层

CXL Memory Pool Allocator + UCX Transport
Metadata Service (全局元数据 + 访问热度)
Tier Policy (热/温/冷分层)

存储层

CXL Pool ($64\text{TB} \times 4 = 256\text{TB}$)
NVMe-oF (1PB+ 归档)
CXL Switch Fabric

三层 API 设计

Layer 1: 基础 CXL Dataset

CXLMemmapDataset: 内存映射 CXL 数据
CXLArrowDataset: Arrow 格式 CXL 数据
CXLNestedDataset: 嵌套/变长数据
CXLWebDataset: 流式 CXL 数据

Layer 2: 高级 Sampler

DistributedCXLSampler: 分布式拓扑感知采样
RandomCXLSampler: 分层随机采样
CurriculumCXLSampler: 课程学习采样
BalancedCXLSampler: 负载均衡采样

Layer 3: 高级 DataLoader

CXLDataLoader: 基础 CXL DataLoader
CXLInstantDataLoader: 即时预取
CXLAsyncDataLoader: 异步流水线
CXLDistributedDataLoader: 分布式 DataLoader

05 关键模块设计

CXLMemoryPool mmap CXL Type 3 Memory Expander

分层分配 (热/温/冷) + madvise 提示

跨节点 RDMA 访问 + cudaHostRegister

CXLMemmapDataset 内存映射 CXL 数据集

分层放置策略 (hot_first/streaming/full)

动态迁移 + 紧凑元数据编码

DistributedCXLSampler 拓扑感知数据分区

分层随机采样 (节点内随机, 节点间顺序)

动态负载均衡

CXLDataLoader pin_to_cxl + prefetch_to_gpu

NUMA 绑定 Worker 池

三级预取流水线 (CXL → DRAM → GPU)

MetadataService 全局元数据服务

访问热度统计 + Bloom filter

Version Control (COW + 版本号)

CXL Memory Pool Allocator 详解

CXLMemoryPool 架构

```
CXLMemoryPool(pool_id='pool_A', size=64TB)
├─ mmap /dev/cxlmem_pool_A
├─ madvise(MADV_HUGEPAGE | MADV_WILLNEED)
├─ CXLBumpAllocator (bump allocator)
```

```
alloc(size, hint='hot', numa_node=0)
├─ hot → MADV_HUGEPAGE | MADV_WILLNEED
├─ warm → MADV_MERGEABLE
├─ cold → MADV_COLD
```

```
pin_for_gpu(buffer)
├─ cudaHostRegister on CXL region
```

```
remote_access(node_id, offset, size)
├─ RDMA Read from remote CXL pool
```

字节级寻址

CXL.mem 直接 load/store
无需 DMA 拷贝

NUMA 感知

绑定到最优 NUMA 节点
避免跨节点访问

分层放置

热/温/冷自动分层
madvise 提示内核

GPU 直访

cudaHostRegister on CXL
GPU P2P 直接访问

CXLMemmapDataset

```
def __init__(self, path, pool,
             tier_policy='hot_first'):
    self.pool = pool
    self.metadata = load_metadata(path)
    self.mmap = pool.mmap_file(path)
    self._tier_placement()

def __getitem__(self, idx):
    offset = self.offsets[idx]
    length = self.lengths[idx]
    # CXL.mem 读取
    data = self.mmap[offset:offset+length]
    return self._decode_sample(data)

def migrate_to(self, tier):
    # 迁移到指定 tier
    pass
```

DistributedCXLSampler

```
def __init__(self, dataset,
             num_replicas, rank, topology):
    self.topology = topology
    self.partition = \
        self._topology_aware_partition()

def _topology_aware_partition(self):
    # 计算能力分数
    scores = {}
    for node in nodes:
        scores[node.id] = \
            node.cxl_bandwidth * \
            node.cxl_capacity
    # 按能力比例分配
    ...

def __iter__(self):
    # 节点内随机, 节点间顺序
    indices = shuffle(local_indices)
    return iter(indices)
```

CXLDataLoader

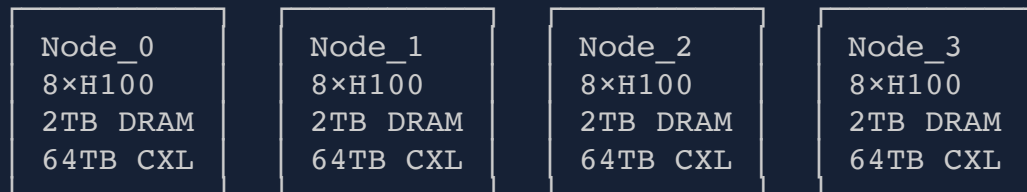
```
def __init__(self, dataset,
             pin_to_cxl=True,
             prefetch_factor=4,
             memory_policy='hot_first',
             numa_bind=True):
    self.pin_to_cxl = pin_to_cxl
    self.prefetch_queue = Queue()

def _pin_to_cxl(self, batch):
    cxl_buffer = self.pool.alloc(
        batch.nbytes, hint='hot')
    memcpy(cxl_buffer.ptr, batch)
    cudaHostRegister(cxl_buffer.ptr)
    return CXLTensor(cxl_buffer)

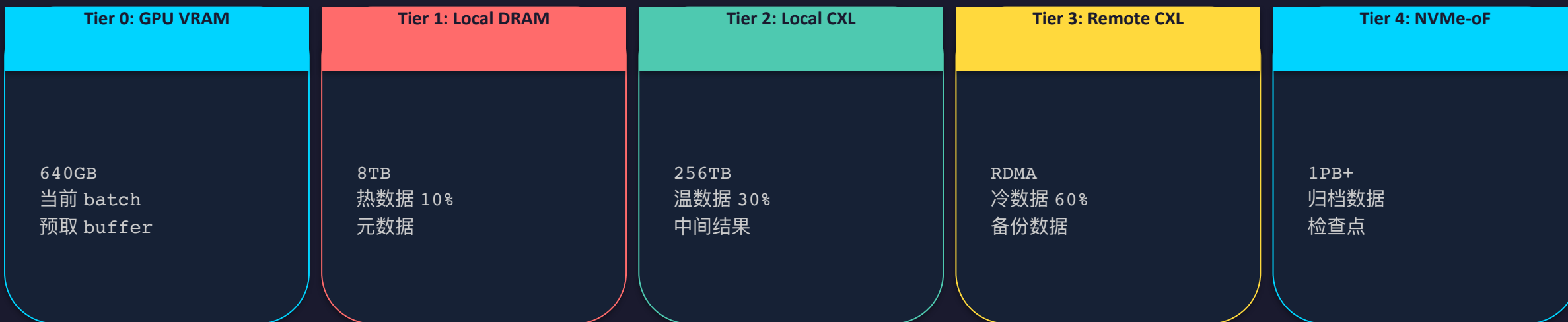
def _prefetch_to_gpu(self, batch):
    # CXL → GPU 异步预取
    return batch.to('cuda',
                    non_blocking=True)
```

06 数百TB DDR内存的驾驭方案

数百 TB CXL 内存池架构



总容量: $4 \times 64\text{TB} = 256\text{TB}$ CXL 内存池 | 总带宽: $4 \times 50\text{GB/s} = 200\text{GB/s}$



三级预取 + 分层全局 Shuffle

三级预取流水线

Level 1: CXL → DRAM (预取线程)

提前 4 个 batch

异步 CXL.mem read

写入 DRAM ring buffer

Level 2: DRAM → GPU (CUDA stream)

提前 2 个 batch

cudaMemcpyAsync

与计算重叠

Level 3: GPU → GPU (GPU Fabric)

提前 1 个 batch

GPU P2P copy

与计算完全重叠

时间线:

```
[CXL Read] [CXL Read] [CXL Read] [CXL Read]
           [DRAM→GPU] [DRAM→GPU] [DRAM→GPU]
                   [GPU Compute] [GPU Compute]
```

分层全局 Shuffle

问题: 100TB 数据无法全量 shuffle

方案: 分层 Shuffle

Level 1: 节点内 shuffle (DRAM)

热数据 (10TB) 全量随机

Level 2: 节点间 shuffle (CXL)

温数据 (30TB) 分块随机

Level 3: 全局 shuffle (RDMA)

冷数据 (60TB) 顺序读取

实现:

1. 将数据分为 1000 个 shard
2. 每个 epoch 重新分配 shard 到节点
3. 节点内 shuffle shard 顺序
4. 节点内随机读取样本

07 性能分析与预期收益

性能模型

设：数据集 $D=100\text{TB}$, $\text{Batch}=32$, $\text{Sample}=1\text{KB}$

CXL 带宽 $\text{BW}_{\text{cxl}}=50\text{GB/s}$, DRAM 带宽 $\text{BW}_{\text{dram}}=100\text{GB/s}$

GPU 计算时间 $T_{\text{comp}}=100\text{ms/batch}$

传统方案 (NVMe)：数据加载 $T_{\text{load}} = D / 10\text{GB/s} = 10000\text{s}$, I/O 占比 $\sim 50\%$

CXL 方案：数据加载 $T_{\text{load}} = D / 50\text{GB/s} = 2000\text{s}$, I/O 占比 $\sim 10\%$

加速比： $\sim 2-5\times$

LLM 预训练

瓶颈：数据加载 I/O

方案：CXL $\rightarrow$ GPU 直传

预期：3-5x

多模态训练

瓶颈：多源数据混合

方案：CXL 分层放置

预期：2-4x

大规模图像训练

瓶颈：图像解码

方案：CXL + GPU 解码

预期：2-3x

RL 训练

瓶颈：环境交互数据

方案：CXL 共享 buffer

预期：1.5-2x

推荐系统

瓶颈：特征工程

方案：CXL 内存池

预期：2-3x

与现有方案的对比

方案	数据规模	跨节点	CXL 感知	全局 Shuffle
PyTorch DataLoader	单节点	✗	✗	✗
FFCV	单节点	✗	✗	✗
MegaScale-Data	多源	✓	✗	✓
DeepSpeed	大规模	✓	✗	✗
CXL DataLoader	数百 TB	✓	✓	✓

CXL DataLoader 是唯一同时支持 数百 TB 数据规模 + 跨节点分布式 + CXL 内存池感知 + 全局 Shuffle 的方案。通过显式利用 CXL 语义（字节级寻址、NUMA 感知、分层放置），在 LLM 预训练场景可获得 3-5x 加速。

Reference Map

数据加载

- 2504.09844 MegaScale-Data: 多源 LFM 训练数据加载, 4.5x 吞吐提升
- 2306.12517 FFCV: 消除数据瓶颈, 20 分钟训练 ResNet-50
- 2212.03597 DeepSpeed Data Efficiency: 12.5x 成本降低, 保持 95% 模型质量

CXL 内存池

- 2602.08800 Equilibria: 多租户 CXL 内存分层, +52% 生产负载
- 2403.18702 NeoMem: 硬件辅助 CXL 分层, +32-67%
- 2312.04789 HybridTier: 自适应轻量 CXL 分层, +91% 峰值
- 2502.19233 HeteroMem: 设备侧透明 CXL 管理, +5.1-16.2%

CXL + ML 训练

- 2606.24079 Aquifer: 分层 CXL+RDMA 内存池, 2.2x MicroVM 启动加速
- 2602.22457 CCCL: CXL 替代 RDMA 的 GPU 集合通信, 1.34-1.94x 提升
- 2411.02282 CXL-DMSim: CXL 解耦内存模拟器, 3.4% 平均误差

结语

通过扩展 PyTorch Dataset/DataLoader API，构建 CXL 内存池感知的分布式数据加载系统，可以显式利用 CXL 语义（字节级寻址、NUMA 感知、分层放置），实现数百 TB 内存的全局数据池化。

关键创新：

- (1) 解耦预处理与 CXL 内存池分配
- (2) 拓扑感知的分布式采样器
- (3) 三级预取流水线 (CXL → DRAM → GPU)
- (4) 分层全局 Shuffle

预期在 LLM 预训练场景获得 3-5x 加速，在推荐系统/多模态训练获得 2-4x 加速。

PyTorch CXL Dataset/DataLoader

驾驭数百TB DDR内存的分布式数据加载架构

Hermes Research | 2026

backyes.github.io

Key References

MegaScale-Data (2504.09844) | FFCV (2306.12517) | DeepSpeed (2212.03597)

Equilibria (2602.08800) | Aquifer (2606.24079) | CCCL (2602.22457)